EEE4120F

High Performance Embedded Systems

Practical 4

Baseline RISC Processor Design in Verilog HDL
The StarCore-1 Mission Computer

Course Team

Course Convener: Associate Professor Simon Winberg

Teaching Assistant: Travimadox Webb

Practical 4

Baseline RISC Processor Design in Verilog HDL
The StarCore-1 Mission Computer

Summary of Deliverables

- **Duration:** 8th April - 20th April (2 weeks)
- **Report:** IEEE Conference Format (PDF, Max 4 Pages) via Gradescope assignment **Practical 4 Report**.
- **Code:** All Verilog source files (.v) and testbenches (*_tb.v) via Gradescope assignment **Practical 4 Oral Exam**.
- **Memory Files:** test.prog and test.data binary initialisation files.
- **Simulation Outputs:** GTKWave waveform screenshots (.png) or VCD files and terminal logs.
- **Oral Examination:** Short Q&A during the lab sessions.

Note

Before starting this practical, complete the companion **Tutorial Document**: *EEE4120F Practical 4 Tutorial Verilog HDL and Icarus Verilog Setup*. It covers tool installation, compilation, simulation, waveform viewing, and core Verilog syntax. A working simulation environment is a mandatory prerequisite.

1 Introduction

This practical introduces Verilog HDL and FPGA-oriented digital design at the register-transfer level (RTL). Verilog (and its successor SystemVerilog) is a Hardware Description Language: a formalism for describing digital circuits at various levels of abstraction. Unlike a software program, a Verilog design does not execute sequentially; rather it describes hardware that operates concurrently. Every `always` block, continuous `assign`, and module instantiation you write exists simultaneously in hardware and evaluates in parallel.

You will design, implement, and simulate a complete 16-bit single-cycle RISC processor, the **StarCore-1**, using Icarus Verilog (iverilog) as your simulation toolchain. No FPGA board is required. All verification will be done entirely in simulation.

This practical serves as the foundation for the course project in which you might extend the StarCore-1 by designing and integrating a custom co-processor that accelerates specialised computations. The reserved opcode `0b1010` in the StarCore ISA is your integration point for that future extension.

On completion of this practical, you will be able to:

1. Describe the Harvard architecture and explain why it simplifies single-cycle processor control.

2. Implement standard digital components such as ALU, register file, memories, control unit — in synthesisable Verilog HDL.
3. Construct a hierarchical Verilog design by composing smaller verified modules into a working processor.
4. Write automated testbenches that exercise all instruction types and produce structured pass/fail output.
5. Interpret simulation waveforms and logs to locate and fix functional bugs in an RTL design.
6. Explain the role of each control signal in the single-cycle datapath and trace the effect of any given instruction through the hardware.

2 Problem Statement

You have been hired as a junior embedded systems engineer at OrbitEdge Technologies, a Cape Town-based aerospace startup developing nano-satellite (CubeSat) platforms for Earth observation and atmospheric sensing. The company is preparing for the launch of its first satellite: OrbitEdge-1.

The OrbitEdge-1 mission requires an on-board Flight Management Computer (FMC) capable of executing pre-programmed command sequences autonomously while orbiting 500 km above the Earth. The FMC must interpret sensor readings from attitude-determination hardware, compute telemetry checksums, and sequence payload operations, all while surviving the extreme radiation environment of low Earth orbit and operating within a tightly constrained power budget of under 400 mW.

The Chief Architect has ruled out commercial off-the-shelf (COTS) processors: radiation-hardened variants are prohibitively expensive, and general-purpose processors carry far more logic than the mission requires, wasting precious power. Instead, the architecture team has specified a custom 16-bit RISC processor core, the **StarCore-1**, to be described in Verilog HDL and implemented on a radiation-tolerant FPGA fabric.

Note

This scenario is intentionally simplified for the practical. Real CubeSat flight computers use triple-modular redundancy (TMR), SEU-hardened flip-flops, formal verification, and often target devices such as the Xilinx Kintex Ultrascale+ or Microsemi RTG4. Your StarCore-1 captures the essential RTL design skills underlying all such systems.
The reserved opcode `1010` in the StarCore ISA is an integration point for a future coprocessor you may design in the course project.

Your Chief Architect has given you the following requirements:

1. The processor must implement the StarCore ISA as defined in Section 3.
2. It must use a Harvard-style architecture with physically separate instruction and data memory buses.
3. All modules must be written in synthesisable, simulation-verified Verilog HDL.
4. Every module must be accompanied by a testbench that exercises all defined behaviours.
5. The integration testbench must demonstrate correct execution of a multi-instruction program that exercises at least load, store, arithmetic, logical, comparison, branch, and jump

instructions.

6. Opcode `1010` (binary) must be handled gracefully by the control unit (treated as a no-operation with no register or memory side-effects) in anticipation of the co-processor extension.

3 The StarCore ISA

The StarCore-1 is a 16-bit, single-cycle, Harvard-architecture RISC processor. Its key properties are:

- 16-bit data and instruction words.
- 8 general-purpose registers, R0–R7, each 16 bits wide.
- Separate instruction memory (ROM, 16 words) and data memory (RAM, 8 words).
- 4-bit opcode field supporting 13 defined instructions plus one reserved co-processor opcode.
- Single-cycle execution: every instruction completes in exactly one clock cycle. There is no pipeline.
- Harvard datapath: the PC fetches from instruction memory while the ALU may simultaneously access data memory, because the two memory buses are independent.

The processor is composed of the following top-level functional units, each of which you will implement as a separate Verilog module:

Module	Role
ALU	Performs all arithmetic and logic operations on 16-bit operands
GPR	8 × 16-bit general-purpose register file (2 read ports, 1 write port)
InstructionMemory	16-word ROM; outputs 16-bit instruction word addressed by PC
DataMemory	8-word RAM; supports synchronous write and conditional read
ALU_Control	Decodes ALUOp + opcode into 3-bit ALU function select
ControlUnit	Decodes 4-bit opcode into all processor control signals
Datapath	Integrates all data-flow components; exposes opcode to ControlUnit
StarCore1	Top-level module connecting Datapath and ControlUnit

3.1 Instruction Word Format

All StarCore-1 instructions are 16 bits wide. There are three instruction formats depending on the instruction type:

Format	Bit Layout [15 : 0]
R-type	[15:12] OP — [11:9] RS1 — [8:6] RS2 — [5:3] WS — [2:0] unused
I-type	[15:12] OP — [11:9] RS1 — [8:6] WS — [5:0] Offset (6-bit signed)
J-type	[15:12] OP — [11:0] Offset (12-bit target)

Field definitions:

- **OP [15:12]** — 4-bit opcode. Decoded by the ControlUnit.
- **RS1 [11:9]** — First source register address (3 bits, selects R0–R7).

- **RS2 [8:6]** — Second source register address (R-type and branch instructions).
- **WS [5:3]** — Write-back destination register address (R-type). For I-type, WS is encoded in [8:6].
- **Offset** — Signed immediate value. 6-bit for I-type (LD/ST/branches), 12-bit for JMP.

Note

The PC register is 16 bits wide and byte-addressed. Because every instruction is 2 bytes (16 bits), the PC always increments by 2. To index into the 16-word instruction memory, use `pc[4:1]` — bits 4 down to 1, discarding the lowest byte-select bit.
Branch offsets are sign-extended from 6 bits to 16 bits and then shifted left by 1 (multiplied by 2) to produce a byte offset, which is then added to PC+2.

3.2 Instruction Set Reference

The StarCore-1 implements 13 instructions. Opcode `1010` is reserved for the co-processor extension. All register operands and results are 16-bit unsigned values unless otherwise stated.

OP	Mnemonic	Syntax	Operation / Semantics
0000	LD	LD WS, offset(RS1)	WS := Mem16[RS1 + SignExt(offset)]
0001	ST	ST RS2, offset(RS1)	Mem16[RS1 + SignExt(offset)] := RS2
0010	ADD	ADD WS, RS1, RS2	WS := RS1 + RS2
0011	SUB	SUB WS, RS1, RS2	WS := RS1 - RS2
0100	INV	INV WS, RS1	WS := ~RS1 (bitwise complement)
0101	SHL	SHL WS, RS1, RS2	WS := RS1 $\ll$ RS2[3:0] (left shift)
0110	SHR	SHR WS, RS1, RS2	WS := RS1 $\gg$ RS2[3:0] (right shift)
0111	AND	AND WS, RS1, RS2	WS := RS1 & RS2 (bitwise AND)
1000	OR	OR WS, RS1, RS2	WS := RS1 — RS2 (bitwise OR)
1001	SLT	SLT WS, RS1, RS2	WS := 1 if RS1 $<$ RS2 (unsigned); else 0
1010	—	(reserved)	Reserved for co-processor. No side-effects.
1011	BEQ	BEQ RS1, RS2, offset	if RS1 == RS2: PC := PC+2 + SignExt(offset)$\ll$1
1100	BNE	BNE RS1, RS2, offset	if RS1 != RS2: PC := PC+2 + SignExt(offset)$\ll$1
1101	JMP	JMP offset	PC := {PC[15:13], offset[11:0], 1'b0}

3.3 Processor Control Signals

The ControlUnit asserts the following signals based on the decoded opcode. This truth table defines the expected output of your ControlUnit module for every instruction type and must be verified in your testbench.

Instruction	RegDst	ALUSrc	MemToReg	RegWrite	MemRd	MemWr	Branch	ALUOp	Jump
LD	0	1	1	1	1	0	0	10	0
ST	0	1	0	0	0	1	0	10	0
R-type	1	0	0	1	0	0	0	00	0
BEQ/BNE	0	0	0	0	0	0	1	01	0
JMP	0	0	0	0	0	0	0	00	1
RSVD(1010)	0	0	0	0	0	0	0	00	0

Signal definitions:

- **RegDst** — Selects the write-back register address: 0 = instr[8:6] (I-type WS field), 1 = instr[5:3] (R-type WS field).
- **ALUSrc** — Selects the ALU B operand: 0 = register RS2, 1 = sign-extended immediate.
- **MemToReg** — Selects write-back data: 0 = ALU result, 1 = data memory read data.

- **RegWrite** — Enables register file write on clock edge.
- **MemRd** — Enables data memory output (asynchronous read).
- **MemWr** — Enables data memory write on clock edge.
- **Branch** — Asserted for conditional branch instructions (BEQ, BNE).
- **ALUOp** — Selects ALU mode: 00 = R-type, 01 = branch comparison, 10 = I-type (ADD).
- **Jump** — Asserted for unconditional jumps (JMP).

4 Tasks

All code is to be implemented in the `src/` directory as specified in Appendix A.

4.1 Task 1: Parameter Definition File

Deliverable

File to implement: `Parameter.v`

Create a shared parameter definition file that all other modules will include. This file should define:

- Word width (16 bits)
- Instruction memory depth (16 words)
- Data memory depth (8 words)
- Simulation duration (adjust as needed for your test program)

Use `define` macros to ensure consistency across all modules.

4.2 Task 2: Arithmetic Logic Unit (ALU)

Deliverable

Module to implement: `ALU.v`

Implement a 16-bit ALU that supports the following 8 operations:

1. Addition (RS1 + RS2)
2. Subtraction (RS1 - RS2)
3. Bitwise AND (RS1 & RS2)
4. Bitwise OR (RS1 — RS2)
5. Bitwise NOT (~RS1)
6. Logical left shift (RS1 $\ll$ RS2[3:0])
7. Logical right shift (RS1 $\gg$ RS2[3:0])
8. Set-on-less-than (RS1 $<$ RS2 ? 1 : 0)

The ALU must also compute a `zero` flag that indicates when the result equals zero. The operation is controlled by a 3-bit `ALUcnt` input.

Tip

Use the three-bit control signal to select operations: e.g., 000 = ADD, 001 = SUB, 010 = AND, 011 = OR, etc. Create a testbench that exercises all 8 operations with known test vectors.

4.3 Task 3: General-Purpose Register File (GPR)

Deliverable

Module to implement: `GPR.v`

Implement an 8-register file with:

- Two asynchronous read ports (read data is available combinationally based on address)
- One synchronous write port (write happens on the positive clock edge)
- Each register is 16 bits wide
- A write-enable signal controls whether a write occurs

Ensure register R0 is always zero (or can be written and then immediately read). Test with read-before-write, write-then-read sequences.

4.4 Task 4: Instruction Memory

Deliverable

Module to implement: `InstructionMemory.v`

Implement a 16-word instruction ROM that:

- Is initialized from a binary file `test/test.prog` using `$readmemb`.
- Outputs a 16-bit instruction word indexed by the PC.
- Uses `pc[4:1]` as the address (discarding the byte-select bits).

4.5 Task 5: Data Memory

Deliverable

Module to implement: `DataMemory.v`

Implement an 8-word data RAM that:

- Is initialized from a binary file `test/test.data` using `$readmemb`.
- Supports a synchronous write on the positive clock edge (when `MemWr` is asserted).
- Supports an asynchronous read (output is combinational, gated by `MemRd`).
- Outputs zero when `MemRd` is not asserted.

4.6 Task 6: ALU Control Unit

Deliverable

Module to implement: `ALU_Control.v`

Implement the ALU Control module that decodes the 2-bit `ALUOp` signal and the 4-bit opcode to generate the 3-bit `ALUcnt` control signal for the ALU:

- ALUOp = 00 (R-type): use the opcode to select the operation (ADD, SUB, AND, OR, INV, SHL, SHR, SLT).

- ALUOp = 01 (Branch): use ALUcnt = 001 (SUB) to compute the difference for branch comparison.
- ALUOp = 10 (I-type): always use ALUcnt = 000 (ADD) for load/store address calculation.

Use a `casex` statement to handle the mapping.

4.7 Task 7: Control Unit

Deliverable

Module to implement: `ControlUnit.v`

Implement the ControlUnit that decodes the 4-bit opcode into all 10 control signals as defined in Section 3:

- RegDst, ALUSrc, MemToReg, RegWrite, MemRd, MemWr, Branch, ALUOp (2 bits), Jump

Use a comprehensive `case` or `casex` statement to decode all 16 possible opcode values. For undefined opcodes, assert no control signals (all outputs = 0).

Warning

The ControlUnit is combinational logic. Use `always @(*)` and blocking assignments (=).

4.8 Task 8: Datapath Integration

Deliverable

Module to implement: `Datapath.v`

Integrate all data-flow components (ALU, GPR, memories, PC logic) into a single Datapath module. The Datapath must:

- Instantiate the ALU, GPR, InstructionMemory, and DataMemory modules.
- Implement the PC register and incrementer (PC+2 logic).
- Route the fetched instruction to an output port for the ControlUnit.
- Implement sign extension for immediate values.
- Implement branch offset calculation and PC-relative branching.
- Multiplex register addresses and ALU operands based on control signals.
- Multiplex write-back data (ALU result or memory read data).
- Handle the `Jump` signal by replacing the upper 3 bits of PC with bits [14:12] of the offset.

Every signal routing decision must be verified by your integration test. Use meaningful signal names and comments.

4.9 Task 9: Top-Level StarCore1 Processor

Deliverable

Module to implement: `StarCore1.v`

Implement the top-level StarCore1 processor that connects the Datapath and ControlUnit. The processor should:

- Instantiate Datapath and ControlUnit.
- Feed the opcode from the datapath to the ControlUnit.
- Route all control signals from the ControlUnit back to the datapath.
- Expose a clock and reset input.
- Optionally expose key signals for debugging (PC, instruction, register writes).

4.10 Task 10: Comprehensive Testbenches

Deliverable

Testbenches to implement: One per module plus integration testbench

Write a testbench for every module you create:

- **ALU_tb.v:** Test all 8 operations with known vectors. Verify the zero flag.
- **GPR_tb.v:** Test read-before-write, write-then-read, register independence.
- **InstructionMemory_tb.v:** Verify correct instruction output for all valid addresses.
- **DataMemory_tb.v:** Verify read/write synchronism, gated output, initialization from file.
- **ALU_Control_tb.v:** Verify all ALUOp and opcode combinations map to the correct ALUcnt.
- **ControlUnit_tb.v:** Verify all 14 valid opcodes (0000–1001, 1011–1101) produce correct control signals. Verify opcode 1010 produces all zeros.
- **StarCore1_tb.v:** Integration test executing a multi-instruction program from test.prog and test.data. The program should exercise load, store, arithmetic, logical, compare, and branch instructions. Print execution trace and waveforms.

5 Report Requirements

Format your report as a technical article in IEEE Conference format. The report must be a PDF document with a maximum of 4 pages and must include all sections listed below. Use professional formatting and avoid pasting raw Verilog code into the report, code belongs in your submitted .v files.

Report Sections

- **Introduction:** Outline the practical objectives, your implementation strategy, the role of the StarCore-1 in the OrbitEdge-1 mission scenario, and the overall design approach (modular decomposition, single-cycle execution, Harvard architecture).
- **Architecture:** Describe the StarCore-1 architecture, including a block diagram of the processor (hand-drawn is acceptable if clear), the Harvard memory organization, the instruction formats, and the role of each major module (ALU, GPR, memories, control unit, datapath).
- **Verification Methodology:** Explain your testbench strategy i.e what each testbench verifies, how you generate test vectors, and how you validate correctness.
- **Results:** Include waveform traces from your integration test and an expected vs observed results table showing at least 5 instruction executions.
- **Discussion :** Analyze the strengths and limitations of your single-cycle design. Discuss the hardware cost (area, power) implications of single-cycle execution compared to pipelined designs. Explain how branch/jump latency is handled. Discuss the reserved opcode 1010: what modifications to the control unit and datapath would be needed to integrate a co-processor? How would you ensure correct arbitration if the co-processor and other functional units compete for the write bus?
- **Conclusion:** Summarise your implementation, state whether your integration test passes end-to-end, identify any remaining limitations, and outline potential future extensions (pipelining, cache, exception handling, co-processor integration).

Figures and Tables

Your report must include:

- A block diagram of the StarCore-1 processor (showing ALU, GPR, instruction and data memories, control unit, PC logic, multiplexers).
- At least one simulation waveform screenshot from GTKWave showing multiple signals for at least 5 clock cycles during program execution (e.g., PC, instruction, register writes, memory operations).
- A table comparing expected vs. observed values for your integration test program (showing at least 5 instruction executions with inputs, outputs, and state changes).

Warning

Do not paste raw Verilog code into the report. Use pseudocode or architectural descriptions instead. Code belongs in your submitted .v files, not in the report.
The 4-page limit is strict. Marks will be deducted for reports that exceed it.

6 Marking Rubric

Category	Component / Criterion	Marks
Report (60%)	Introduction	5
	Architecture Overview	5
	Verification Methodology	10
	Results	15
	Discussion	15
	Conclusion	5
	Formatting	5
Oral Exam (40%)	Verilog Fundamentals	8
	StarCore-1 ISA and Instruction Encoding	8
	Processor Architecture and Control Signals	8
	Verification and Debugging	8
	Design Analysis and Extension	8
Total		**100**